

Writeup: Vicious

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_15_ViciousOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `LMA0_NOT_THEFL4G`

Résumé

Write-up Vicious

etape 1 : Identification du binaire

```
file Vicious
```

etape 2 : Localiser la section .data

Il y a 7 sections headers, offset 0x2368:

Outils Utilisés

On désassemble tout :

```
objdump -d Vicious > dump.asm
```

first_half_loop – 8 itérations, XOR simple :

```
0000000000401067 <first_half_loop>:
401067: 67 8a 86 54 20 40 00  mov  0x402054(%esi),%al
40106e: 67 8a 9e 2f 20 40 00  mov  0x40202f(%esi),%bl
401075: 30 d8                xor  %bl,%al
401077: 67 3a 86 3f 20 40 00  cmp  0x40203f(%esi),%al
40107e: 0f 85 86 00 00 00    jne  40110a <fail>
401084: ff c6                inc  %esi
401086: e2 df                loop 401067 <first_half_loop>
401088: b9 08 00 00 00      mov  $0x8,%ecx
40108d: 31 f6                xor  %esi,%esi
```

Conclusion

```
input[i] ^ key_table1[i] == enc_flag1[i]
//key_table1 & enc_flag1 font chacun 8 octets.
```

modify_key_table2 – incrément de la deuxième clé :

```
40108f: 67 8a 86 37 20 40 00  mov  0x402037(%esi),%al
401096: fe c0                inc  %al
```

```
401098:  67 88 86 37 20 40 00  mov  %al,0x402037(%esi)
40109f:  ff c6                inc  %esi
4010a1:  e2 ec                loop 40108f <modify_key_table2>
```

Tous les octets de key_table2 prennent +1 (8 fois).

second_half_loop – XOR + ADD 5

```
4010aa:  67 8a 86 5c 20 40 00  mov  0x40205c(%esi),%al
4010b1:  04 05                add  $0x5,%al
4010b3:  67 8a 9e 37 20 40 00  mov  0x402037(%esi),%bl
4010ba:  30 d8                xor  %bl,%al
4010bc:  67 3a 86 47 20 40 00  cmp  0x402047(%esi),%al
4010c3:  75 45                jne  40110a <fail>
4010c5:  ff c6                inc  %esi
4010c7:  e2 e1                loop 4010aa <second_half_loop>
```

La relation trouvée :

$(enc_flag2[i] == (input[i+8] + 5) \text{ XOR } (key_table2[i] + 1))$

checksum_loop – somme des 16 octetss :

```
4010d2:  67 8a 87 54 20 40 00  mov  0x402054(%edi),%al
4010d9:  0f b6 c0             movzbl %al,%eax
4010dc:  01 c2                add  %eax,%edx
4010de:  ff c7                inc  %edi
4010e0:  e2 f0                loop 4010d2 <checksum_loop>
4010e2:  3b 14 25 4f 20 40 00  cmp  0x40204f,%edx
```

magic_checksum est un dword = 4 octets.

Analyse Statique

Section Headers:

Name	Type	Address	Offset
Size	EntSize	Flags Link Info	Align
NULL	0000000000000000	00000000	
0000000000000000	0000000000000000		0 0 0
.text	PROGBITS	0000000000401000	00001000
0000000000000138	0000000000000000	AX	0 0 16
.data	PROGBITS	0000000000402000	00002000
0000000000000053	0000000000000000	WA	0 0 4

Début de .data : 0x2000 dans le fichier.

Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

Identification du Mécanisme de Validation

Valeur lu dans le dump : 0x00000488 → 1160.

etape 5 découper correctement la section .data
Grâce aux tailles vues dans l'assembleur :

bloc Octets dans le dump (xxd)

key_table1 (8 o) 12 34 56 78 9a bc de f0
key_table2 (8 o) 0f 1e 2d 3c 4b 5a 69 78
enc_flag1 (8 o) 5e 79 17 48 c5 f2 ee a4
enc_flag2 (8 o) 74 46 63 77 07 0a 53 35
magic_checksum (4 o) 88 04 00 00

etape 6 : reconstituer le flag

8-1 Première moitié

flag1[i] = enc_flag1[i] XOR key_table1[i]

i enc_flag1 key_table1 Résultat (ASCII)
0 0x5e 0x12 0x4c L
1 0x79 0x34 0x4d M
2 0x17 0x56 0x41 A
3 0x48 0x78 0x30 0
4 0xc5 0x9a 0x5f _
5 0xf2 0xbc 0x4e N
6 0xee 0xde 0x30 0
7 0xa4 0xf0 0x54 T

flag1 = "LMA0_NOT"

Découverte du Flag

Étape 3 : Dumper les données brutes

```
xxd -s 0x2000 -l 0x53 Vicious

00002000: 476f 6f64 204a 6f62 210a 0042 6164 2050  Good Job!..Bad P
00002010: 6173 7377 6f72 6421 0a00 464c 4147 7b46  assword!..FLAG{F
00002020: 414b 4546 4c41 4731 3233 3435 367d 0012  AKEFLAG123456}..
00002030: 3456 789a bcde f00f 1e2d 3c4b 5a69 785e  4Vx.....-<KZix^
00002040: 7917 48c5 f2ee a474 4663 7707 0a53 3588  y.H....tFcw..S5.
00002050: 0400 00                                     ...
```

Reconstitution de la première moitié du flag

```
flag1 = []
for i in range(8):
    decrypted_byte = enc_flag1[i] ^ key_table1[i]
    flag1.append(chr(decrypted_byte))
```

```
print("".join(flag1))
```

Conclusion

```
Conclusion
input[i] ^ key_table1[i] == enc_flag1[i]
//key_table1 & enc_flag1 font chacun 8 octets.
```

Longueur

Longueur : 0x53 = 83 octets.

Étape 4

Étape 4 : Désassembler le code critique :

```
readelf -s Vicious | grep -E 'first_half_loop|second_half_loop|checksum_loop|
modify_key_table2'
```

12:	00000000000401067	0	NOTYPE	LOCAL	DEFAULT	1	first_half_loop
13:	0000000000040108f	0	NOTYPE	LOCAL	DEFAULT	1	modify_key_tabl
							e2
14:	000000000004010aa	0	NOTYPE	LOCAL	DEFAULT	1	second_half_lo
							op
15:	000000000004010d2	0	NOTYPE	LOCAL	DEFAULT	1	checksum_loop

Deuxième moitié

Deuxième moitié :

i enc_flag2 key_table2+1 temp temp-5 ASCII

0	0x74	0x10	0x64	0x5f	_
1	0x46	0x1f	0x59	0x54	T
2	0x63	0x2e	0x4d	0x48	H
3	0x77	0x3d	0x4a	0x45	E
4	0x07	0x4c	0x4b	0x46	F
5	0x0a	0x5b	0x51	0x4c	L
6	0x53	0x6a	0x39	0x34	4
7	0x35	0x79	0x4c	0x47	G

flag2 = "_THEFL4G"

checksum

La somme ASCII de LMA0_NOT_THEFL4G vaut 1160, du coup c'est valide.

Script python pour calculer cela

Script python pour calculer cela :

```
enc_flag2 = [0x74, 0x46, 0x63, 0x77, 0x07, 0x0a, 0x53, 0x35]
key_table2 = [0x0f, 0x1e, 0x2d, 0x3c, 0x4b, 0x5a, 0x69, 0x78]
flag2 = []
```

```
for i in range(8):
    key_plus_1 = key_table2[i] + 1
    temp = enc_flag2[i] ^ key_plus_1
    flag_byte = temp - 5
    flag2.append(chr(flag_byte))

print("".join(flag2))

flag_1 :
```

Données extraites de .data

Données extraites de .data

```
key_table1 = [0x12, 0x34, 0x56, 0x78, 0x9a, 0xbc, 0xde, 0xf0]
enc_flag1 = [0x5e, 0x79, 0x17, 0x48, 0xc5, 0xf2, 0xee, 0xa4]
```